



PDF Download
3719345.pdf
28 January 2026
Total Citations: 9
Total Downloads:
1487

Latest updates: <https://dl.acm.org/doi/10.1145/3719345>

RESEARCH-ARTICLE

ContrastRepair: Enhancing Conversation-Based Automated Program Repair via Contrastive Test Case Pairs

Published: 03 October 2025
Online AM: 04 March 2025
Accepted: 04 February 2025
Revised: 26 December 2024
Received: 03 March 2024

[Citation in BibTeX format](#)

JIAOLONG KONG, Singapore Management University, Singapore City, Singapore

XIAOFEI XIE, Singapore Management University, Singapore City, Singapore

MINGFEI CHENG, Singapore Management University, Singapore City, Singapore

SHANGQING LIU, Nanjing University, Nanjing, Jiangsu, China

XIAONING DU, Monash University, Melbourne, VIC, Australia

QI GUO, Tianjin University, Tianjin, China

Open Access Support provided by:
Singapore Management University
Nanjing University
Monash University
Tianjin University

ContrastRepair: Enhancing Conversation-Based Automated Program Repair via Contrastive Test Case Pairs

JIAOLONG KONG, XIAOFEI XIE, and MINGFEI CHENG, Singapore Management University, Singapore, Singapore
SHANGQING LIU, Nanjing University, Nanjing, China
XIAONING DU, Monash University, Melbourne, Australia
QI GUO, Tianjin University, Tianjin, China

Automated Program Repair (APR) aims to automatically generate patches for rectifying software bugs. Recent strides in Large Language Models (LLM), such as ChatGPT, have yielded encouraging outcomes in APR, especially within the conversation-driven APR framework. Nevertheless, the efficacy of conversation-driven APR is contingent on the quality of the feedback information. In this article, we propose *ContrastRepair*, a novel conversation-based APR approach that augments conversation-driven APR by providing LLMs with contrastive test pairs. A test pair consists of a failing test and a passing test, which offer contrastive feedback to the LLM. Our key insight is to minimize the difference between the generated passing test and the given failing test, which can better isolate the root causes of bugs. By providing such informative feedback, *ContrastRepair* enables the LLM to produce effective bug fixes. The implementation of *ContrastRepair* is based on the state-of-the-art LLM, ChatGPT, and it iteratively interacts with ChatGPT until plausible patches are generated. We evaluate *ContrastRepair* on multiple benchmark datasets, including Defects4J, QuixBugs, and HumanEval-Java. The results demonstrate that *ContrastRepair* significantly outperforms existing methods, achieving a new state-of-the-art in program repair. For instance, among Defects4J 1.2 and 2.0, *ContrastRepair* correctly repairs 143 out of all 337 bug cases, while the best-performing baseline fixes 124 bugs.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**;

Additional Key Words and Phrases: Program Repair, Large Language Model

ACM Reference format:

Jiaolong Kong, Xiaofei Xie, Mingfei Cheng, Shangqing Liu, Xiaoning Du, and Qi Guo. 2025. *ContrastRepair: Enhancing Conversation-Based Automated Program Repair via Contrastive Test Case Pairs*. *ACM Trans. Softw. Eng. Methodol.* 34, 8, Article 216 (October 2025), 31 pages.
<https://doi.org/10.1145/3719345>

This research is supported by the National Research Foundation, Singapore, and the Cyber Security Agency under its National Cybersecurity R&D Programme (NCRP25-P04-TAICeN). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore and Cyber Security Agency of Singapore.

Authors' Contact Information: Jiaolong Kong, Singapore Management University, Singapore, Singapore; e-mail: jlkong@smu.edu.sg; Xiaofei Xie (corresponding author), Singapore Management University, Singapore, Singapore; e-mail: xfxie@smu.edu.sg; Mingfei Cheng, Singapore Management University, Singapore, Singapore; e-mail: mfcheng.2022@smu.edu.sg; Shangqing Liu (corresponding author), Nanjing University, Nanjing, China; e-mail: shangqingliu@nju.edu.cn; Xiaoning Du, Monash University, Melbourne, Australia; e-mail: xiaoning.du@monash.edu; Qi Guo, Tianjin University, Tianjin, China; e-mail: bxguoqi@tju.edu.cn.



This work is licensed under Creative Commons Attribution International 4.0.

© 2025 Copyright held by the owner/author(s).

ACM 1557-7392/2025/10-ART216

<https://doi.org/10.1145/3719345>

1 Introduction

With the increasing complexity of software, the presence of bugs and vulnerabilities has become inevitable. These issues can lead to system failures, security breaches, and a compromised user experience. Manually debugging and fixing these problems is a time-consuming and laborious task, demanding substantial resources and effort from developers. As reported, the annual expenditure on bug finding and fixing amounts to billions of dollars [4], and developers spend about 50% of their time in the crucial process of debugging and fixing errors [3, 29, 82]. In light of this challenge, **Automatic Program Repair (APR)** has emerged as a promising solution, offering automated generation of patches to fix bugs.

APR has been the subject of extensive research in recent years. Traditional APR techniques can be categorized into template-based [35, 36, 47], heuristic-based [31, 32, 64], and constraint-based [30, 41, 48] methods. However, the effectiveness of these traditional methods is still not satisfactory. For instance, template-based APR requires manually designed templates, demanding significant human effort and domain knowledge. Consequently, such methods exhibit limited generalization capability, primarily functioning well only on specific types of bugs they were designed for. Recently, machine learning techniques, particularly **Deep Learning (DL)**-based APR, have gained prominence. These DL models have the advantage of learning diverse patterns of buggy problems from a vast amount of data, surpassing the performance of traditional methods. Nonetheless, DL-based APR techniques still face certain challenges. One significant concern is the reliance on training data; if the training data lacks representation of certain bug types, the model may still struggle to generalize to unseen bugs effectively [69, 70]. Moreover, constructing comprehensive bug-fixing datasets for training the DL models requires substantial effort and resources. Furthermore, while DL-based methods show promise in APR, their effectiveness remains limited. These approaches often produce a considerable number of candidate patches, leading to a time-consuming validation process, which presents significant obstacles to the practical implementation of these techniques [24].

To overcome the challenges, a promising approach that has gained traction in more recent research is the use of **Large Language Models (LLMs)**. LLMs are trained on vast datasets, such as large-scale code corpora, and have demonstrated superior performance across various tasks [10, 24, 63, 69, 70]. The primary factor for their superior performance lies in the remarkable ability to comprehend program semantics. In the context of APR, studies [24, 56, 60, 67, 68] have shown that even without fine-tuning, LLMs exhibit competitive fixing capabilities compared to traditional DL-based APR techniques. The fixing capability can be significantly enhanced by fine-tuning the LLMs on relevant data [24]. Previously, most of the works invoke LLMs independently without incorporating conversation. With the advance of dialog-based LLMs, such as ChatGPT, recent conversational-driven APR based on ChatGPT has achieved new state-of-the-art performance [69, 70]. The basic idea is to generate patches in a conversational style. At each conversation, the system combines previous incorrect patches with test failure information to prompt the LLM to generate a new patch. This *conversational-driven* APR has demonstrated remarkable performance in guiding the LLM to propose more effective repairs.

This article primarily focuses on the utilization of LLMs for *conversation-driven* APR. The paramount challenge in applying LLMs for this purpose is the formulation of high-quality prompts, which play a crucial role in guiding the LLM to comprehend programs and subsequently fix bugs. While recent research [70] has shown promising results using LLMs, the proposed approach mainly depends on providing error feedback information for the conversations. Unfortunately, this feedback might not always offer specific and informative prompts for effective repair.

To this end, we propose a method aiming to craft more specific and informative prompts for enhancing the capabilities of LLMs in accurately localizing bugs and generating high-quality

fixes. Our key insight lies in that relying solely on negative feedback, i.e., error information from failing tests, may not always be adequate for LLMs to precisely pinpoint the bugs. We propose the inclusion of positive feedback derived from successful tests to supplement the negative feedback. By providing comparative input pairs that juxtapose the outcomes of these tests, LLMs are more likely to effectively localize the root cause. We propose a novel conversation-driven approach *ContrastRepair* that generates prompts by utilizing both positive and negative feedback. Given a failing test case and the buggy function, we create a corresponding passing test case quite similar to the failed one, forming a contrastive pair that is then forwarded to the LLMs.¹ By providing such pair-wise information, LLMs receive more specific and informative cues, allowing them to better pinpoint the root cause of the bug and generate accurate fixes. Nevertheless, a challenge lies in selecting the appropriate passing test case (from a considerable number of passing tests) to pair with the failing case. Our basic idea is to construct the test pair (i.e., the failing test and the passing test) by minimizing their difference, such that the difference (which makes one test fail and another one pass) can isolate and pinpoint the failure causes. Specifically, we present a method designed to generate a passing test case by making minimal modifications to a failing one. If there already exists a collection of passing test cases, we then select the most similar one from the set. In this process, we devise a metric (based on edit distance) to measure the similarity score, a threshold is used to decide whether the constructed test pair will be retained. For bugs without a passing test, we design a series of mutation strategies to mutate the failing tests and generate new passing tests. Then we try to pair the failing tests with these newly generated passing tests. Additionally, some bugs have strong dependencies on other functions, which is also an important context for the repair. We incorporate the contextual information by selecting relevant functions from the traceback of the failing test case. Finally, the prompt provided to the LLM consists of the *buggy function*, the *dependent functions*, the *test pair*, and the *traceback information* from the failing test case. The LLM then outputs a patch function. If the patch is deemed incorrect, we adopt a similar approach to iteratively generate the next patches. This iterative process continues until plausible patches, i.e., those capable of passing all the tests, are produced, or the repair budget is exhausted.

We evaluated *ContrastRepair* on three distinct datasets: Defects4J, QuixBugs, and HumanEval-Java. Specifically, we compared the performance of *ContrastRepair* against state-of-the-art approaches, encompassing both learning-based APR methods and conversation-based APR methods, such as CHATREPAIR [70]. The results of our evaluation clearly demonstrate that *ContrastRepair* significantly outperforms the baseline methods. Notably, *ContrastRepair* successfully resolved 360 out of 581 bugs, surpassing the state-of-the-art ChatGPT-based method CHATREPAIR, which could fix 334 bugs. Additionally, our findings reveal that *ContrastRepair* exhibits enhanced efficiency in terms of API calls with an average reduction of 20.91% compared to CHATREPAIR. Furthermore, our ablation study underscored the significance of pair selection and contextual information in augmenting the performance of *ContrastRepair*.

In summary, this article makes the following contributions:

- We present a novel approach for APR that leverages LLMs in a conversation-driven manner. Through the integration of both negative and positive feedback, our approach empowers LLMs to provide high-quality fixes.
- We propose a method to construct and prioritize suitable test pairs comprising both failed and corresponding passing tests. This method ensures that the prompt provided to LLMs is more informative, aiding in precise bug localization and more effective patch generation.

¹ While our approach is general, we specifically chose ChatGPT in this work due to its state-of-the-art performance in handling conversational-style interactions.

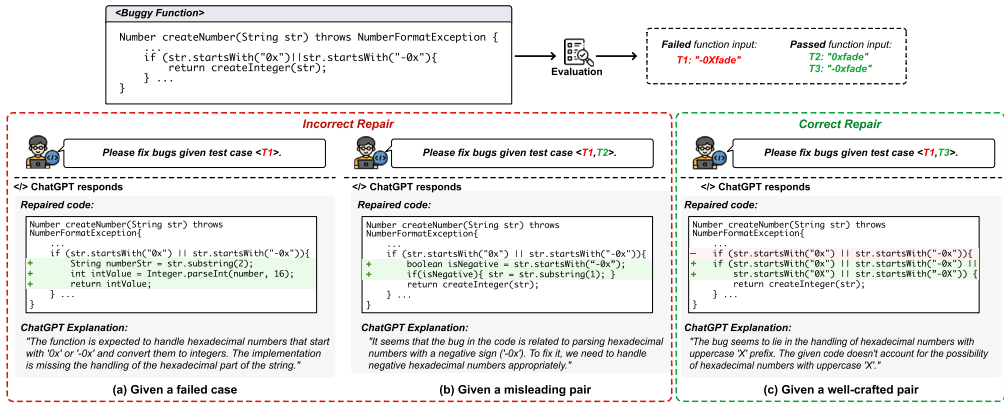


Fig. 1. A motivating example.

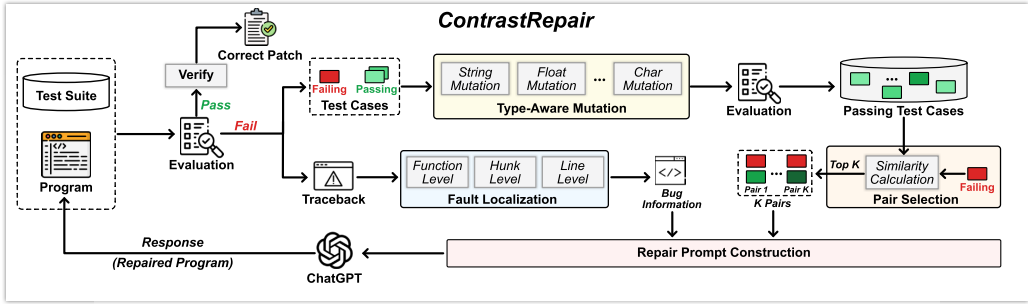
— We perform a comprehensive evaluation to evaluate the effectiveness of our tool. The results demonstrate that *ContrastRepair* achieves new state-of-the-art performance in terms of correct bug fixes, increasing by 15.32% than the best baseline, surpassing existing APR tools. All experimental data and source code are publicly available [28].

2 Motivation Example

Figure 1 depicts a motivating example (Lang-16 from Defects4J) that comprises a buggy function and a set of test cases designed for evaluating this function. By giving different types of test cases, we show the results of ChatGPT, including fixed code and the explanation about the fixes. Specifically, `createNumber` works to parse a given string `str` and returns its corresponding `Number` object. A subtle bug is present in the function code. When `str` represents the string of a hexadecimal integer, the function correctly handles cases where it starts with lowercase "0x" or "-0x". However, it overlooks the uppercased inputs, leading to incomplete handling of corner cases. Consequently, when evaluating `createNumber` on three test cases, both T2 and T3 pass, while T1 fails.

When fixing this program, the repairer needs to first locate the buggy line(s) and then correctly address the missed corner case. Relying on ChatGPT, although it exhibits a good understanding of program semantics, additional guidance on the requested task remains beneficial. While the failing test case T1 is supplied to aid in the repair process and narrow down the search space by providing hints on the buggy lines and potential fixes, the impact of failing test cases alone is somewhat limited. As a result, more effective guidance is desired to achieve better results.

On the other hand, when we provide a well-crafted contrastive test pair $\langle T1, T3 \rangle$, such as ("0xfade", "-0xfade"), it significantly simplifies the task for the repairer to deduce that the characters "X" and "x" may be the problematic elements causing the bug. This enables the repairer to accurately pinpoint the buggy lines, i.e., those dealing with problematic elements, and propose a suitable fix. It shows how contrastive information derived from both positive and negative feedback can offer more precise guidance for program repair. By leveraging such contrastive test pairs, the repair process becomes more efficient and effective. However, it is crucial to recognize that not every test pair provides valuable guidance for the repair process. In some cases, the guidance from certain comparisons may be indirect or even incorrect. For instance, the pair $\langle T1, T2 \rangle$, i.e., ("0xfade", "0xfade"), could lead ChatGPT to mistakenly infer that the symbol "-" is the cause of the bug, which could be misleading for the repair. Thus, *how to better select and craft contrastive test pairs is of great importance to ensure the effectiveness of the repair guidance.*

Fig. 2. Overview of *ContrastRepair*.

3 Methodology

3.1 Overview of *ContrastRepair*

Figure 2 provides an overview of the *ContrastRepair*. Essentially, it involves a conversation process that constructs prompts using contrastive test pairs, feeds these prompts to an LLM such as ChatGPT, and receives responses that lead to the repaired code.

Specifically, at each iteration of the conversation, *ContrastRepair* takes as input a program to be validated, along with a test suite consisting of multiple test cases. The initial program is assumed to be buggy code. *ContrastRepair* then evaluates the program with the test suite. If all tests pass successfully, a plausible patch is obtained, which is further investigated by a human. On the other hand, if any test fails (f), and triggers the bug in the program, then the corresponding traceback logs (T_f) are captured. Additionally, all passing tests (P) that the program handles correctly are also collected for further test pair construction.

Next, *ContrastRepair* constructs test pairs that involve the failing test f and other passing tests. To ensure these test pairs provide contrast guidance, we propose a similarity-guided selection process. Inspired by the concept of Delta Debugging [79], which aims to isolate failure causes by minimizing the *failing test*, we select a passing test that has minimal difference from the failing test. This enables ChatGPT to isolate failure causes by contrasting their minor differences. In cases where no existing test cases have high similarity with f , we introduce a type-aware test mutation technique to generate a satisfactory passing test based on its type. It minimizes the mutation of the failing test case, thereby enhancing the contrastive information that is helpful in isolating the failure causes.

We then follow the best practices [2] to construct prompts for ChatGPT. To collect the dependency information of the bug, we identify the dependent functions from the traceback of the failing test case. The prompt consists of the traceback logs from the failing test case f , the constructed test pairs, the dependent functions, the target buggy function, and the description of the requirements. With this prompt, ChatGPT generates the repaired code, which is then validated in the subsequent iteration. The iterative process continues until a plausible patch is identified or the repair budget (e.g., the maximum number of iterations) is reached.

3.2 Constructing Contrastive Test Case Pairs

A common sense for code debugging is that “Often people who encounter a bug spend a lot of time investigating which changes to the input file will make the bug go away and which changes will not affect it” [79]. The changes that make the bug go away are called *critical changes*. These critical changes hold valuable insights into understanding the root cause of bugs, making them essential

for successful bug localization and repair. Therefore, our objective is to identify and provide critical change information to LLMs.

To facilitate the exposure of the crucial changes to LLMs, we present a method that involves constructing contrastive test case pairs, consisting of a failing test case and a corresponding passing test case. We ensure that the failing test and the passing test are sufficiently similar, which allows LLMs to deduce critical changes based on their minor differences leading to triggering the bug.

Specifically, to create the contrastive test case pairs, we first construct a set of passing tests P . Then we select the suitable passing tests (denoted as S) from P to pair with the given failing f , ensuring that they exhibit higher similarity with f : $S = \{p | p \in P \wedge \delta(f, p) > \theta\}$, where δ represents the similarity measurement between the failing test f and a passing test p , and θ is a predefined threshold. Two main problems need to be addressed: how to measure the similarity (δ) and how to construct the passing test set (P).

3.2.1 Similarity Measurement (δ). Various similarity metrics can be considered. However, since our primary objective is to expose critical changes regarding the failing test, a string-based similarity metric is more relevant for this task. In this article, we select the *Damerau-Levenshtein distance* [13, 33] as the chosen similarity measure. The Damerau-Levenshtein distance is a metric used to gauge the similarity between two strings by quantifying the minimum number of operations required to transform one string into the other. It accounts for four types of edit operations, i.e., insertion, deletion, substitution, and transposition. A smaller Damerau-Levenshtein distance indicates greater similarity between the two cases, which is more beneficial for LLMs to isolate the root causes. We normalize the similarity score to the range (0, 1):

$$\delta(f, p) = \frac{1}{n} \sum_{i=1}^n 1 - d_i / \max(\text{len}(f_{arg_i}), \text{len}(p_{arg_i}))$$

where n represents the number of arguments in the test case, arg_i means the i th argument, d_i indicates the distance between the i th argument for failing case (f) and passing case (p), and $\text{len}()$ is utilized to calculate the length of the argument which is parsed into String type. Taking the test cases in Figure 1 as an example, there is only one argument in the buggy function, thus n is 1. The value of failing test T1 is “-0xfade,” and the values of passing test T2 and T3 are “0xfade” and “-0xfade,” respectively. The Damerau-Levenshtein distance calculated between T1 and T2 is 2, and between T1 and T3 is 1. Thus we can have $\delta(T1, T2) = 1 - 2/7 = 5/7$, $\delta(T1, T3) = 1 - 1/7 = 6/7$.

Note that the test cases may have different types. To accommodate this, we convert the tests with other types into string representation before calculating their similarity using the Damerau-Levenshtein distance. For object or array types, we recursively convert each element to its corresponding string representation and then calculate the similarity.

3.2.2 Passing Test (P). To gather the passing test cases, we adopt two strategies: utilizing existing passing tests and generating new passing tests. Many projects, such as Defects4J, include unit tests that are designed to validate the correctness of individual functions. Specifically, we consider the test case as the input to the buggy function. To obtain these inputs, we perform the instrumentation by applying the Java lib-Javassist [11] to instrument code at the start of the buggy function. This allows us to capture the parameter values when each unit test from the test suite is run by the original test driver supplied by developers. We execute all unit tests and collect each function’s corresponding passing test cases. In cases where certain functions lack unit tests or the existing unit tests have low similarity with the failing test f , we propose a type-aware mutation technique to generate new passing test cases. The mutation is based on the failing test f with changes made as minimally as

possible. When testing the mutation-generated test cases, we also employ instrumentation to the buggy function by altering the original arguments to mutated ones.

Type-Aware Mutation. The type-aware mutation comprises two main components: a general string-based mutation and type-specific mutations. Given that the similarity is measured based on string representations, the natural approach is to first convert the failing test f to a string representation. We then perform string-level mutations while constraining the degree of mutations. Finally, we convert the string mutants back to their original types based on a Python lib-javaobj [6]. Additionally, we employ type-specific mutations to enhance the diversity of the generated test cases. The mutation strategies are listed as follows:

- *String Mutation:* (1) Random Character Replacement; (2) Random Substring Replacement; (3) Random Character Insertion; (4) Random Character Deletion; (5) Substring Swapping; (6) Case Conversion and (7) Truncation/Extension.
- *Integer, Double, and Float Mutation:* (1) Random Perturbation, adding or subtracting a small random value; (2) Scaling, multiplied by a scaling factor; (3) Flip Sign, changing from positive to negative or vice versa and (4) Magnitude Perturbation, adding or subtracting a small percentage of the number's magnitude.
- *Char Mutation:* Replace the original character with a randomly chosen different character (Random Replacement).
- *Boolean Mutation:* Negate the Boolean value (Negation).
- *Object Mutation:* Element-wise mutation is based on the mutations in primitive types above.
- *Array, List:* (1) Element-wise Mutation, mutating one or more elements; (2) Element Swapping; (3) Element Insertion; (4) Element Deletion and (5) Element Shuffling.

As functions may have multiple parameters, we consider each parameter as an element or attribute of an object. Consequently, we treat the set of parameters as an object-type test case. This allows us to conduct object-level mutation and similarity comparisons.

Test Oracle. The collection of passing test cases poses a challenge due to the lack of test oracles, especially for logical bugs. We categorize bugs into two groups: exception bugs, for which the test oracles can be determined based on whether the exception is thrown, and logical bugs, which can only be captured through explicit assertions. For exception bugs, we employ type-aware mutation to generate passing tests. However, for logical bugs, in this article, we primarily rely on existing assertion-based test cases to serve as our test oracles for logical bugs. If there is no existing passing test case, we use a single failing test instead of a pair. We leave the integration and exploration of existing solutions for addressing logical bug oracles (e.g., constructing metamorphic relations) as our future work.

Algorithm 1 demonstrates the procedure for generating passing test cases by mutating multiple arguments. The inputs to this algorithm consist of a failing test case f , the total number of arguments m within the test case, and the maximum number *budget* in terms of the mutation. The algorithm outputs a set of passing test cases ranked by their similarity. The core approach involves randomly selecting a number of arguments n and mutating their values within a range from 1 to m (Lines 4–7). Specifically, for each selected argument, we apply the type-aware mutation to generate a new test, as described earlier. To constrain the mutation, we set a mutation budget *budget* as the maximum allowable limit. After performing the mutations, the similarity between each variant in *MCases* and the failing case f is calculated (see Section 3.2.1) (Line 8). Finally, failing test cases are filtered out, and the remaining passing tests are ranked based on their similarity scores (Line 9).

Algorithm 1: *PassingCaseAugment*

Input: f : the failing test case, m : the number of arguments in test case, $budget$: the maximum number of mutations

Output: $PCases$: the newly-constructed passing cases from mutation

```

1  $MCases := \emptyset;$  // a set to store all the newly-constructed cases from mutation
2  $PCases := \emptyset;$  // a set to store all the newly-constructed passing cases from mutation
3  $n := 1;$ 
4 while  $n \leq m$  do
    // generate test case variants from the failing case  $f$  by mutating  $n$  out of  $m$  arguments
5      $TmpMCases := \text{Mutate}(f, n, budget);$  // generate test case variants by mutating  $n$  arguments in  $f$ 
6      $MCases := MCases \cup TmpMCases;$  // add  $TmpMCases$  to  $MCases$ 
7      $n := n + 1;$ 
8  $SimScores := \text{Similarity}(f, MCases);$  // collect all the similarity scores
9  $PCases := \text{FilterandRank}(SimScores, MCases);$  // Filter failing tests and rank the passing ones
10 return  $PCases;$  // return passing test cases constructed by mutation

```

3.3 Conversation-Driven Repair

The repair process follows an iterative approach that involves interacting with LLMs to gradually repair the buggy function. Two main strategies can be employed during the repair process:

- *Continuing from Previous Patch:* In this strategy, at each iteration, we continue the repair process from the previous patch that is still not correct. This approach allows us to build on the previous repair attempts and leverage continuous feedback from LLMs to refine and improve the patches incrementally. However, there is a risk that the repair process may continue in the wrong direction, potentially worsening the patches.
- *Restarting Repair from Original Buggy Function:* Alternatively, at each iteration, we can start the repair process from the original buggy function. This strategy ensures that each repair attempt begins from a clean state, reducing the risk of compounding errors from previous iterations. However, it also loses the continuous feedback that can be valuable for refining the repair.

To strike a balance between these two strategies, we impose a limit on the depth of continuous repairs. If the function cannot be repaired within the specified maximum depth, we restart the repair process from the original buggy function and select different test pairs. This approach allows us to explore multiple repair paths and increase the chances of generating accurate fixes.

Algorithm 2 presents the overall repair process of *ContrastRepair*. The algorithm takes as input the buggy function and a set of test cases α used to evaluate its correctness. The objective is to output a plausible patch if successful. *ContrastRepair* collects all failing test cases and passing test cases by running all available tests (Line 1). In cases where the number of passing tests is insufficient, or they have low similarities with the failing test, the function *Collect* includes the type-aware mutation to generate additional passing tests for exception bugs. Then it selects all contrastive test pairs that exhibit sufficient similarity (Line 2). The repair process involves both restarting the repair (Line 4) and continuous repair (Line 8). The total number of repair attempts is limited to $m \times n$, where m is the maximum tries for the restarting repair and n is the maximum tries for the continuous repair.

At each restarting iteration, k test pairs are selected for constructing prompts (Line 5). The *SelectPair* function aims to prioritize pairs based on how often a pair is selected before. The pairs that were rarely selected have high priority, which prevents an excessive number of selections

Algorithm 2: *ContrastRepair*

Input: *Func*: the buggy function, α : the whole tests, k : the number of pairs selected, θ : the threshold of similarity score,
 m : the maximum number of restarting repair attempts, n : the maximum number of continuous repair attempts
Output: *PFuncs*: the plausible patch(es)

```

1   $P, F := \text{Collect}(\text{Func}, \alpha);$  // collect passing and failing tests
2   $\phi := \{(f, p) | f \in F \wedge p \in P \wedge \delta(f, p) > \theta\};$  // get all similar pairs
3   $iter_1 := 0;$  // initiate the number of restarting repair attempts
4  while  $iter_1 < m$  do
5       $\rho := \text{SelectPair}(\phi, k);$  // restart repair from original function // select  $k$  pairs
6       $tmp := \text{Func};$ 
7       $iter_2 := 0;$  // initiate the number of continuous repair from previous patch
8      while  $iter_2 < n$  do
9           $T := \text{Execute}(tmp, \rho_f);$  // continuous repair from previous patch // get traceback info
10          $D := \text{ExtractDF}(\text{Func}, T);$  // get dependent functions from  $T$ 
11          $\text{prompt} := \text{ConstructPrompt}(tmp, \rho, T, D);$  // construct prompt for the next query
12          $\text{Func}' := \text{ChatGPT}(\text{prompt});$  // invoke ChatGPT and get the new patch
13          $P', F' := \text{Collect}(\text{Func}', \alpha);$  // evaluate the patched function
14         if  $F' = \emptyset$  then
15              $PFuncs := \text{PatchAug}(\text{Func}')$  // generate alternative plausible patches
16             return  $PFuncs;$  // plausible patches
17          $\phi' := \{(f, p) | f \in F' \wedge p \in P' \wedge \delta(f, p) > \theta\};$ 
18          $\rho := \text{SelectPair}(\phi', k);$ 
19          $tmp := \text{Func}';$  // update  $\phi'$ ,  $\rho$  and  $tmp$ 
20          $iter_2 = iter_2 + 1;$ 
21      $iter_1 = iter_1 + 1;$ 
22 return  $\text{Func};$  // fail to repair

```

of the same pairs. For the selected pairs ρ , the algorithm performs continuous repair within n tries. The failing test cases in the k pairs (denoted as ρ_f) may or may not be caused by the same root cause. We execute all of them to collect the bug traceback logs and remove the redundant duplicates to reduce the tokens sent to LLMs (Line 9). In cases where the bug depends on other functions, *ContrastRepair* identifies the dependent functions from the traceback logs, then merges them together and forms the dependent function collection D (Line 10). The prompt is constructed (Line 11), including the buggy function tmp , the test pairs ρ , the traceback t , dependent functions d , and specific repair requirements. The prompt is then fed to ChatGPT, and the patched function is obtained (Line 12).

The patch is evaluated with all tests (Line 13). If passed, a plausible patch is generated. Following the work [70], we generate alternative plausible patches by ChatGPT in case the generated patch is not the correct patch but is very close to it (Line 15). Otherwise, the repair process continues from the current patched function and its test pairs (Lines 17-19). If the repair fails within the given budget, the original buggy function is returned.

Prompt Construction. The construction of prompts is a crucial step in effectively guiding ChatGPT for program repair. Following best practices [2, 20, 65], we design the prompt to include essential elements that provide context, task description, and constraints on the output. We manually

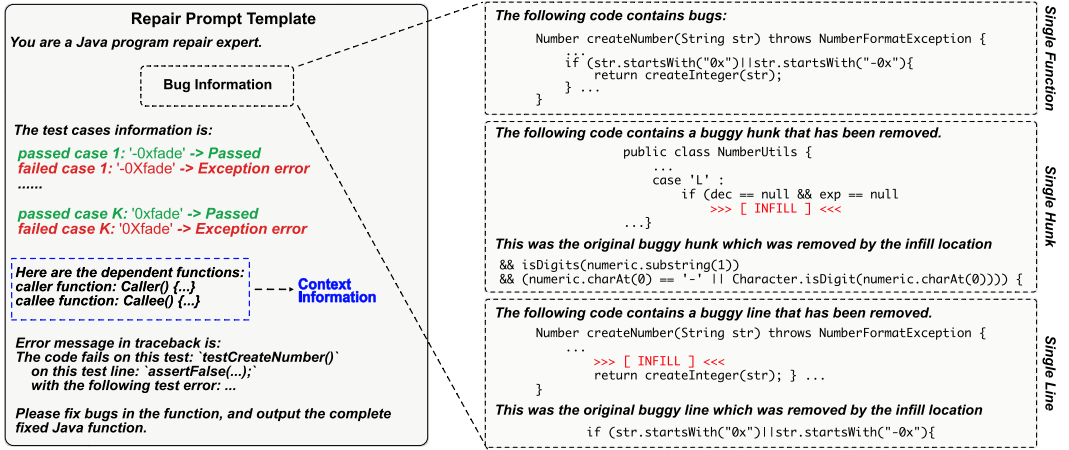


Fig. 3. Illustration of the prompt construction.

examined several alternative prompts using the web version of ChatGPT and selected the best one. As is presented in Figure 3, the prompt begins by setting the context for ChatGPT, introducing it as a “Java program repair expert.” Then it includes the code of the buggy function that requires repair, along with all the chosen contrastive test pairs that were constructed. The test failure diagnosis included in traceback information from the failed test is added to the prompt, providing additional context and insights into the bug. If the bug depends on other functions, we include the code about these dependent functions. Specifically, we extract the function names that appear in both the traceback of the error message and the source code of the project. Those that are direct callers or callees of the buggy function are considered dependent functions. If a dependent function depends on other functions, then others are also taken into account unless the length of the prompt hits the maximum limit. The dependent function could provide more information for ChatGPT to understand the buggy program and the root causes of bugs. The prompt concludes with specific requirements, instructing ChatGPT to generate a complete patched function.

4 Evaluation

In this section, we aim to answer the following research questions:

- RQ1: How effective is *ContrastRepair* compared to state-of-the-art APR techniques? We evaluate the performance of *ContrastRepair* by comparing it to traditional APR methods, DL-based APR techniques, and the recent conversation-driven APR tools.
- RQ2: How useful is *ContrastRepair* in unknown datasets? Considering the potential data leak risk caused by LLMs, we evaluate *ContrastRepair* on previously unseen datasets, which were not used during the training of ChatGPT.
- RQ3: How do different hyper-parameters affect the repair performance of *ContrastRepair*? We conduct a study to examine the impact of various hyper-parameters on *ContrastRepair*’s effectiveness, including the number of test pairs, and the thresholds for continuous repair and restarting repair.
- RQ4: What are the contributions of different components of *ContrastRepair* in improving repair effectiveness? We aim to understand the usefulness of each component of *ContrastRepair*, including the contrastive test pairs and the dependent functions.

4.1 Setup

4.1.1 Configuration. For the experiments conducted in this article, we opted to utilize ChatGPT with the gpt-3.5-turbo-0301 model as the pre-trained LLM for *ContrastRepair*. To interact with the ChatGPT service, we employed its API. In order to increase the potential search space and generate diverse patches, we followed the settings in ChatRepair by setting the sampling temperature to 1 while using the default setting of GPT-3.5 for the seed value. The maximum number of continuous repair attempts (n) and the maximum number of restarting repair attempts (m) were set as 3 and 40. Therefore, the maximum number of conversations per buggy function is 120. Specifically, in the process of querying ChatGPT to repair bugs, there is no timeout setting, when query times are used up, the repair process stops. Once a plausible patch is generated, we take patch augmentation to generate more plausible patches by prompting ChatGPT with already collected plausible ones, and we set query times for this process to 40. Based on an initial study on similarity distribution after mutations (see RQ1), we set the similarity threshold, θ , at 0.5. This decision balances the number of test pairs we can select while aiming to maximize their similarity. Instead, we generated a set of test cases by mutating the failing test from a small to a large degree or by collecting existing passing tests. For the mutation of the failing test, we randomly generate 1,000 new test cases for validation and selection. For evaluating newly generated test cases, there is a timeout. Specifically, we set the maximum time as 30 seconds to evaluate each generated test case and verify whether the buggy function can pass it. Notably, in our approach, test generation is only invoked once before the repair process starts.

4.1.2 Benchmark. We evaluate the effectiveness of *ContrastRepair* by using well-established benchmark datasets, including Defects4J and QuixBugs. Defects4J is a widely studied collection of open-source bugs from 17 different projects, while QuixBugs consists of 40 buggy and fixed versions of classic programming problems in both Python and Java.

To provide fault localization during the repair process, we adopt the approach used in prior research [68, 70] by providing perfect fault locations derived from ground truth. We evaluate the performance of the tools in three different scenarios based on the granularity of the provided fault locations: at the function level, the statement level, and the hunk level. These scenarios correspond to single-function fixes, single-line fixes, and single-hunk fixes, respectively. It is worth noting that function-level fault localization is more practical and more challenging for repair, given that fine-grained fault localization is often difficult to obtain in real-world situations.

For the Defects4J dataset, we follow the standard practice in prior APR works and divide it into two versions: Defects4J 1.2 and Defects4J 2.0. Defects4J 1.2 contains a total of 391 bugs in six different Java projects. Based on the fault localizations provided, they can be categorized as 255 single-function bugs, 154 single-hunk bugs, and 80 single-line bugs. Due to the budget constraint for invoking the ChatGPT API, for Defects4J 2.0, we select 82 single-line bugs, which is a commonly used setting in prior APR tools [68, 70] for easier comparison. Notably, the category of single-line bugs is a subset of the single-function bug category and single-hunk bug category, and the single-hunk category is also a subset of the single-function category. Additionally, we also assess the performance of *ContrastRepair* on QuixBugs dataset, which consists of both Java and Python bugs.

Additionally, for our evaluation, we incorporated the HumanEval-Java [24] benchmark, which has 163 bug cases and most of them are single-line ones. The release date of HumanEval-Java postdates the data collection used for training GPT-3.5, ensuring that there is less risk of data leakage. Within this benchmark, developers translated Python programs from HumanEval and their corresponding test cases into Java programs and JUnit test cases. Some bugs are introduced into the correct Java programs. Similarly, we also provide function-level localization for the repair of all bugs.

Table 1. Bug Proportion on Each Benchmark

Dataset	% Assertion	% Exception
Defects4J	67.16%	32.84%
QuixBugs-Python	65.00%	35.00%
QuixBugs-Java	62.50%	37.50%
HumanEval-Java	45.40%	54.60%

For the benchmarks mentioned above, we have calculated the respective proportions of assertion-type and exception-type bugs within each benchmark, allowing us to know about the scope within which mutations can be implemented. As is shown in Table 1, the results indicate that in most benchmarks, the proportion of assertion-type bugs is higher than that of exception-type bugs. This is particularly obvious in Defects4J, where assertions account for 67.16%, which means we can only mutate 32.84% of bugs among all the 335 bugs.

4.1.3 Implementation. In our selected datasets, the provided unit tests may not always target the buggy functions; they might instead target the callers of these buggy functions. To obtain test cases specifically designed for the buggy functions, we employ an instrumentation process. Specifically, we use the Java lib-Javassist [11] to insert instrumentation code at the beginning of the buggy functions to capture the parameter values during the execution of unit tests. These captured values effectively serve as test cases tailored for the buggy functions. Once we have gathered the test cases, we apply our type-aware mutation strategy to the failing tests, resulting in a set of mutated test cases. For executing the mutated test cases, we utilize the original test driver employed by Defects4J. During the execution of the test driver, the instrumented code within the buggy functions substitutes the values of the parameters with the values from the mutated test cases.

4.1.4 Baselines. In our comparative evaluation, we evaluate *ContrastRepair* by comparing it with seven state-of-the-art baselines, including five learning-based APR methods (SelfAPR [75], AlphaRepair [68], RewardRepair [76], Recoder [84], and CURE [25]), one traditional APR method (TBar [36]), and one recent LLM-based technique, CHATREPAIR [70]. Additionally, we create an LLM-based baseline called BaseChatGPT, which uses a basic prompt without detailed feedback, such as test pairs and traceback information. BaseChatGPT takes the buggy function as input and iteratively outputs the patched functions. Specifically, the only component retained by BaseChatGPT is the buggy function, which is included in the *prompt* and will be updated by the newly generated function from the last response during the multiple iterations of the conversations. We selected only one traditional APR method because: (1) TBar is the state-of-the-art traditional method and (2) previous research [42, 68, 70, 75, 84] has demonstrated that their performance is usually inferior to learning-based methods.

We chose CHATREPAIR due to two main reasons: first, both *ContrastRepair* and CHATREPAIR adopt a conversation-based paradigm for program repair, making the comparison more relevant. Second, CHATREPAIR has achieved new state-of-the-art results, as reported in [70], surpassing the state-of-the-art methods.

4.1.5 Metrics. We select two widely used metrics to compare *ContrastRepair* with the baselines:

- *Number of Correct Fixes (#Correct)*: Assesses the ability of the repair tool to produce accurate patches. This metric counts the number of programs that have been properly repaired based on a manual review of the plausible patches generated by each tool.

Table 2. Comparative Results on Defects4J. Bold values indicate the best performance among all the tools.

Dataset	<i>ContrastRepair</i>	BaseChatGPT	CHATREPAIR	AlphaRepair	SelfAPR	RewardRepair*	Recoder	TBar*	CURE
Chart	12	10	11	8	9	5	11	11	9
Closure	32	19	30	22	18	15	22	16	13
Lang	19	10	17	11	12	7	9	13	9
Math	30	23	26	21	15	19	22	22	16
Mockito	8	5	5	5	2	3	1	3	4
Time	2	1	1	3	3	1	2	3	1
D4J1.2	103	68	90	70	59	50	67	68	52
Cli	4	3	5	5	6	3	3	-	2
Codec	5	4	6	5	6	4	3	-	5
Collections	0	0	0	0	0	0	0	-	1
Compress	2	2	2	1	2	2	2	-	1
Csv	3	2	2	2	1	1	3	-	0
Gson	1	1	2	2	0	1	0	-	0
JacksonCore	3	3	3	3	3	2	2	-	2
JacksonDatabind	7	5	4	6	7	5	1	-	2
JacksonXml	1	1	1	1	1	1	0	-	0
Jsoup	14	7	9	10	5	5	4	-	5
Jxpath	0	0	0	1	0	1	0	-	1
D4J2.0	40	28	34	36	31	25	18	8	19
Total correct/plausible	143/201	96/143	124/178	106/155	90/119	75/-	85/135	76/120	71/133

— *Number of Queries to ChatGPT (#Query)*: Quantifies the resource utilization and efficiency of the LLM-based method. We evaluate the average number of ChatGPT API queries made across all bug cases, providing insight into the frequency with which ChatGPT calls are needed.

4.2 RQ1: Effectiveness of *ContrastRepair*

Repair Performance on Defects4J. Table 2 presents the results of different tools on the number of correct fixes for Defects4J 1.2 and Defects4J 2.0, where the tools are represented by abbreviation. Notably, *ContrastRepair*, BaseChatGPT, and CHATREPAIR’s results on Defects4J 1.2 encompass the union of single-function fix, single-hunk fix, and single-line fix. The symbol * indicates that the results were not obtained by us but rather collected from their paper [70] due to the code not being open-sourced or the failure to run in our local configuration. However, the configuration of them are the same as that of the other baselines, and we firmly believe that this does not affect the validity of our comparative conclusions, as *ContrastRepair* consistently demonstrates significant improvements compared to these baselines. As the results suggest, *ContrastRepair* achieves the best performance compared to other baselines. Specifically, *ContrastRepair* can correctly repair 103 out of 255 bugs and 40 out of 82 bugs in Defects4J 1.2 and Defects4J 2.0, respectively. It can also generate 201 plausible fixes out of the total 335 bugs, accounting for 60.00% among them. On Math and Jsoup projects, *ContrastRepair* significantly outperforms the other baselines, fixing 15.38% and 40.00% more bugs, respectively, than the second best-performing baseline. These results set a new state-of-the-art repair performance, showcasing the effectiveness of *ContrastRepair*.

Comparing the LLM-based tools with other methods, we can clearly observe that integrating ChatGPT significantly improves the repair performance. For instance, the best non-LLM method, AlphaRepair, correctly repairs 70 and 36 bugs on Defects4J 1.2 and 2.0, respectively. On the other hand, BaseChatGPT can achieve competitive performance, i.e., correctly repairs 68 and 28 bugs, demonstrating the usefulness of LLM in the repair task.

Comparing *ContrastRepair* with other LLM-based methods, we can observe that both *ContrastRepair* and CHATREPAIR achieve better results than BaseChatGPT, highlighting the significance of providing informative feedback to LLM in program repair. Note that, there is a gap between the results of CHATREPAIR obtained in this article and that in the original paper [70]. The main

Table 3. Comparison with Baselines on QuixBugs. Bold values indicate the best performance among all the tools.

QuixBugs	<i>Contrast Repair</i>	CHAT REPAIR	Alpha Repair	Reward Repair	CURE
Python	40	38	27	-	-
Java	40	37	28	20	26

Table 4. Comparison on Distinct Scenarios. Bold values indicate the best performance among the three LLM-based methods.

Tools	D4J1.2			QuixBugs-Py			QuixBugs-J		
	SL	SH	SF	SL	SH	SF	SL	SH	SF
BaseChatGPT	38	50	37	31	32	30	29	30	33
CHATREPAIR	45	53	52	33	35	34	32	33	34
<i>ContrastRepair</i>	56	69	75	36	36	37	34	35	37

Table 5. Comparison with Baselines on D4J1.2. Bold values indicate the best performance among the three LLM-based methods.

Tools	#Correct			#Plausible			#Query		
	SL	SH	SF	SL	SH	SF	SL	SH	SF
BaseChatGPT	38	50	37	43	67	67	66.88	79.89	96.16
CHATREPAIR	45	53	52	47	70	101	62.36	79.14	82.27
<i>ContrastRepair</i>	56	69	75	60	102	120	44.38	55.24	77.36

reason, leading to that the results of CHATREPAIR reported in the original paper are superior, is the variation of hyper-parameter settings. In the original paper, the default setting for the maximum number of restarting repair attempts allowed is 200 for single-line and single-hunk scenarios, and 100 for the single-function scenario, which are much larger than ours. A further comparison between *ContrastRepair* and CHATREPAIR is presented in Table 5, where columns SL, SH, and SF show the results of single-line fixes, single-hunk fixes, and single-function fixes, respectively. As we can see, among all the fix scenarios, *ContrastRepair* performs better than the others. In the three repair scenarios of Defects4J 1.2, *ContrastRepair* significantly outperforms CHATREPAIR by repairing 11, 16, and 23 more bugs, respectively. In the single-line-fix scenario of Defects4J 2.0, *ContrastRepair* outperforms CHATREPAIR with 40 correct bug fixes compared to 34. Furthermore, we found that *ContrastRepair* generates about 20% more plausible fix than CHATREPAIR, 56 versus 47. The improvements demonstrated that *ContrastRepair* could provide more useful prompts, such as the test pairs, which contributes to its superior performance.

Repair Performance on QuixBugs. Table 3 displays the results for QuixBugs. Notably, *ContrastRepair* successfully rectified all bugs within the QuixBugs-Java and QuixBugs-Python, demonstrating its marked superiority over other methods. Detailed repair results across three distinct scenarios are presented in Table 4. Specifically, *ContrastRepair* outperforms CHATREPAIR among three scenarios in both QuixBugs-Python and QuixBugs-Java. In QuixBugs-Python, *ContrastRepair* can respectively fix three, one and three more bugs than CHATREPAIR in three different scenarios. Similarly, QuixBugs-Java can perform better by giving two, two and three more correct fixes.

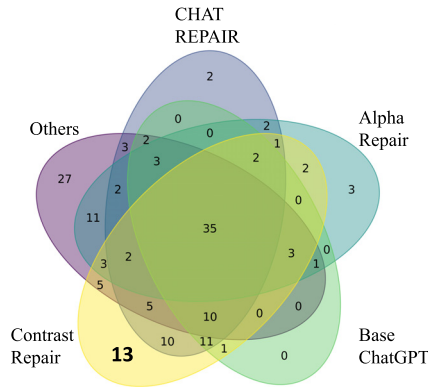


Fig. 4. Bug fix Venn diagram on D4J1.2.

Repair Efficiency. The average monetary cost associated with *ContrastRepair* is indeed slightly higher than that associated with CHATREPAIR as there is more information in the prompt. For example, in the single-line repair scenario within Defects4J, the average cost of correctly fixing a bug using our method is \$0.38, compared to \$0.32 with CHATREPAIR. On the other hand, *ContrastRepair* has improved repair efficiency significantly in terms of the number of API calls, which is reported in Table 5. Compared to CHATREPAIR, *ContrastRepair* exhibited an enormous reduction in average query times among all bug cases (*#Query*), especially in single-line and single-hunk repair scenarios, with *#Query* decreased by 28.83% and 30.20%, respectively. It tends to indicate that *ContrastRepair* employs better prompts, allowing the LLM to more quickly grasp the cause triggering the bug with fewer attempts. In the single-function repair scenario, we noticed that *#Query* only decreased by 5.97%. After analyzing the experiment results, we found that *ContrastRepair* repaired some bugs that CHATREPAIR was unable to fix. Patches of these bugs are more complicated, *ContrastRepair* could only address them when it approached the maximum number of attempts, leading to a marginal decrease in *#Query*.

Figure 4 shows a Venn diagram of the bugs fixed by all baselines and *ContrastRepair* on Defects4J 1.2. We selected the top three baselines based on the number of bugs they fixed correctly and grouped the remaining ones under “Other.” We can see that *ContrastRepair* fixes 13 unique bugs not addressed by any other prior tools.

Figure 5 presents a specific example (Lang-27) that was only fixed by *ContrastRepair*. This bug manifested as an unexpected `StringIndexOutOfBoundsException` error on the highlighted line. In Figure 5(a), we showcase the buggy function and highlight the line where failing cases trigger the unexpected exception. The input is considered passed if it either parses successfully or raises a `NumberFormatException` when it is invalid. As depicted in Figure 5(b), CHATREPAIR erroneously directs ChatGPT’s focus toward the value of `expPos`, leading to an incorrect adjustment in its calculation. When provided with the failing test case `1eE`, *ContrastRepair* initially generates a similar passing test case, `eE1`. We conjecture that the passing test could possibly imply that the calculation of `expPos` is fine and a proper fix can be attained by adding a bounds check. Additionally, the values of `expPos` for the passing test `eE1` and the failing test `1eE` are `expPos=2` and `expPos=4`, respectively, with both inputs having a length of 3. This contrastive information ($4 > 3$ and $2 < 3$) might provide stronger guidance for ChatGPT to infer that *the bug is triggered when the value of expPos exceeds the input length*. As a result, as shown in Figure 5(c), *ContrastRepair* successfully fixes the bug.

Figure 6 presents a case study, specifically Jsoup-40, that demonstrates the necessity of contextual information for a successful repair where CHATREPAIR fails to correct the issue. The figure is

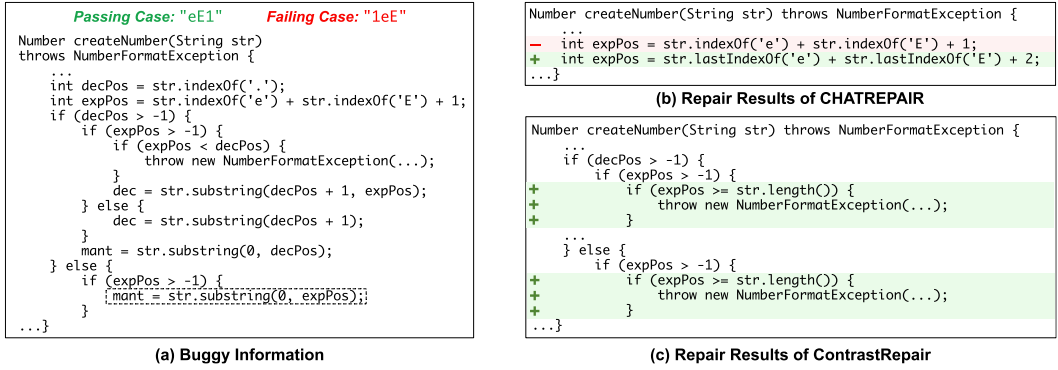


Fig. 5. An illustrative example of a bug uniquely fixed by *ContrastRepair* in Defects4J 1.2.

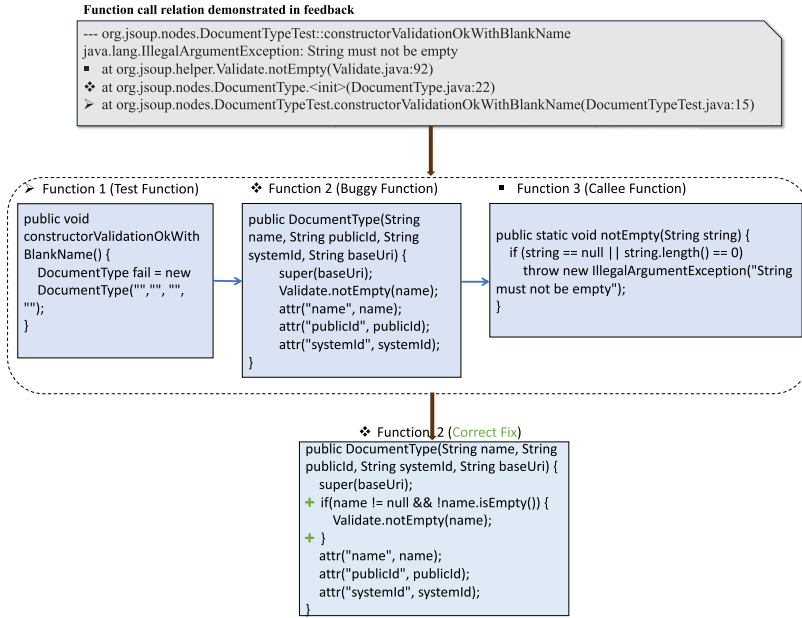


Fig. 6. An illustrative example of the context helping repair.

structured vertically into three parts: the traceback log at the top, the source code of functions triggered in the log in the middle, and the correct fix of the buggy function at the bottom. In this instance, the buggy function is `DocumentType`, and it relies on the function `notEmpty`. The issue arises when an empty `String` `name` is input, causing the dependent function `notEmpty` to trigger an `IllegalArgumentException`. By analyzing the relevant source code of the dependent function in our approach, *ChatGPT* can identify the root cause of the bug and implement a safety check to ensure that `name` is neither `null` nor empty. As a result, *ContrastRepair* successfully repairs the bug.

Performance of Type-Aware Mutation. We also evaluate the effectiveness of the proposed type-aware mutation. We find that when applying the mutation operations, we have 45 bugs in Defects4J dataset that can successfully generate passing test cases to form newly constructed pairs. The average similarity scores for these pairs, based on different mutation rules, are as follows: (1)

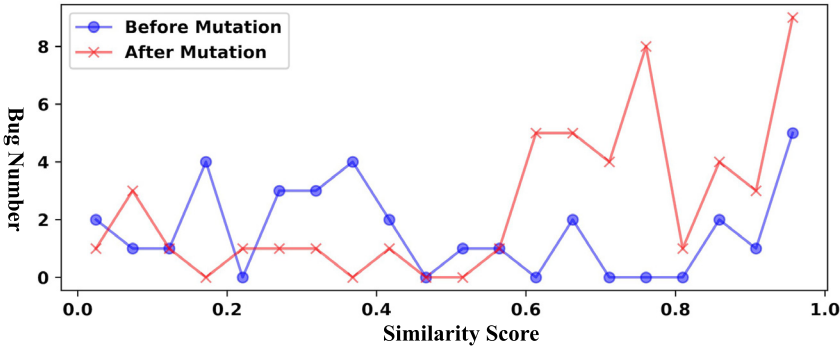


Fig. 7. Similar scores before and after mutation.

String: 0.89; (2) Integer, Double, and Float: 0.72; (3) Char: 0.81; (4) Boolean: 0.78; (5) Object: 0.91; (6) Array, List: 0.79. The results show that pairs resulting from mutations on the Object type have the highest average similarity score. This is because Objects typically have multiple attributes, and modifying only one while keeping the rest unchanged results in a minor overall change, thus a higher similarity score. Conversely, for types like Integer, Double, or Float, even a small random addition or subtraction results in a significant edit distance, leading to a lower similarity score. The remaining bugs we cannot successfully generate passing test cases are because these buggy functions have no parameters in primitive type or all parameters are complicated objects instantiated from classes defined by developers. We also analyzed the test pair similarity before and after type-aware mutation. Figure 7 displays the frequency distribution of maximum similarity scores before and after mutation. The x-axis denotes similarity scores, while the y-axis represents the number of bugs. For each bug, we calculate the highest similarity score among its multiple test pairs. Note that a bug receives a maximum similarity score of 0 if it has no test pairs. The original number of this kind of bug is 43, however, after mutation, it decreases to 25. The results reveal a wide distribution of initial similarity scores, with only 14.67% (11/75) of the bugs falling between 0.5 and 1. However, after mutation, these scores predominantly cluster in the 0.6 to 1 range, encompassing 53.33% (40/75) of the bugs. This indicates a notable enhancement in similarity scores attributable to our mutation strategy. For each bug, we randomly generate 1,000 test cases and validate them via the command provided by Defects4J.

Answer to RQ1: ContrastRepair improves the state-of-the-art APR tools significantly, including ChatGPT-based repair method CHATREPAIR. For instance, in the three fix scenario of Defects4J 1.2, ContrastRepair achieves a success repair rate of 70.00%, 44.81%, and 29.41%, respectively, whereas CHATREPAIR achieves 56.25%, 34.42%, and 20.39%, respectively. Furthermore, ContrastRepair requires fewer API queries than CHATREPAIR.

4.3 RQ2: Evaluation on Unknown Dataset

In light of concerns surrounding potential data leaks from the widely used benchmarks, Defects4J and QuixBugs, which could be included in the training data of ChatGPT, we conducted experiments to assess the effectiveness of *ContrastRepair* on unseen new datasets. It is worth noting that while there is a possibility of data influence from Defects4J and QuixBugs, our comparative analysis with ChatRepair and BaseChatGPT in RQ1 still underlines the effectiveness of *ContrastRepair*, given that all three methods employ the same version of ChatGPT. To explore a fresh perspective, we opted to utilize the recently introduced benchmark, HumanEval-Java, which was made publicly

Table 6. Comparison on HumanEval-Java. Bold values indicate the best performance among the three LLM-based methods.

Tools	#Plausible	#Correct	#Query
BaseChatGPT	136	126	28.90
CHATREPAIR	143	130	24.68
<i>ContrastRepair</i>	151	137	17.44

Table 7. Evaluation on Pair Number. Bold values indicate the best performance among all the settings.

Settings	#Fail	#Pair	#Correct	#Plausible	#Query
CHATREPAIR	0	0	33	54	69.09
BaseChatGPT	0	0	31	48	76.16
+SingleFail	1	0	32	53	69.23
+DoubleFail	2	0	33	55	68.57
+SinglePair	0	1	35	58	66.14
<i>ContrastRepair</i>	0	2	37	60	63.09

available in January 2023. In our evaluation, *ContrastRepair* was configured for single-function fixes with setting n and m as 3 and 40, and we compared the outcomes with those obtained using CHATREPAIR and BaseChatGPT.

As illustrated in Table 6, *ContrastRepair* achieves the best results among the three tools, successfully fixing 137 out of 164 total bugs. In comparison, CHATREPAIR achieved 130 fixes, while BaseChatGPT managed 126. Notably, *ContrastRepair* exhibited an improvement in the success repair rate by 5.38% and 8.73% when compared to CHATREPAIR and BaseChatGPT, respectively.

Furthermore, *ContrastRepair* significantly outperforms the other two methods in terms of efficiency, as indicated by the average number of ChatGPT queries on each bug (#Query). Specifically, *ContrastRepair* reduced the number of #Query by 29.34% (from 24.68 to 17.44) in comparison to CHATREPAIR, and by 39.66% (from 28.90 to 17.44) when compared to BaseChatGPT. These results are consistent with our findings from the Defects4J benchmark, demonstrating its effectiveness.

Answer to RQ2: The evaluation of the unseen dataset, HumanEval-Java, again showcases the superior performance of *ContrastRepair* in terms of both the number of correct fixes and the efficiency of ChatGPT API queries.

4.4 RQ3: Hyper-Parameter Evaluation

The primary hyper-parameters for *ContrastRepair* encompass the number of test case pairs (k) and the total number of attempts, determined by the continuous and restarting repair configuration (m, n). We perform the single-function repair as it is a more practical scenario, and the dataset consists of 106 bugs (from Defects4J) in which at least one pair can be constructed. Table 7 shows the outcomes from various test case configurations (Columns #Fail and #Pair). These configurations span four variants: a single failed case, two failed cases, one test pair, and two test pairs. We present the repair results of LLM-based baselines on these 106 bugs. We limit the maximum number of test pairs to 2, as half of the bugs in the dataset have fewer than two pairs of test cases with a similarity threshold of 0.5. For consistency, we only retrain using the top-2 test pairs when a bug has more

Table 8. Evaluation of the Impact of Increasing Iteration Depth. Bold values indicate the best performance among all the settings.

Settings		ContrastRepair		
Iteration (m)	Round (n)	#Correct	#Plausible	#Query
40	1	26	39	27.89
40	2	33	54	44.99
40	3	37	60	63.09
40	3 (with the same prompt)	35	58	65.08

than two pairs. Comparisons were made based on the number of correct fixes (Column #Correct), plausible fixes (Column #Plausible), and ChatGPT API queries (Column #Query).

Our observations revealed that *ContrastRepair*, when provided with two test case pairs, yielded the highest correct and plausible fix rates (37/60). Offering just a single test case pair led to the second-highest fix effectiveness (35/58). Crucially, using test case pairs consistently outperformed merely submitting the same quantity of failed test cases. Moreover, introducing two failed test cases (33/55) was more effective than just one (32/53). Notably, *ContrastRepair*, when fed with two test case pairs, demanded the fewest #Query (63.09). These findings underscore the value of test pairs: they deliver more useful information to LLMs for bug repair than simply relaying failed tests. Consequently, supplying an increased number of tests or test pairs tends to optimize results.

To assess the effectiveness of continuous repair (i.e., whether the repair direction across n iterations tends to be correct), we consider four configurations, including 40×1 , 40×2 , 40×3 , and 40×3 (with the same prompt). Note that, unlike the first three configurations, the last one, i.e., 40×3 (with the same prompt), uses the identical prompt across all n queries. By comparing this with the 40×3 configuration that uses continuous prompts, we can determine whether the repair direction across the n iterations is correct and useful. Comparing the results among these three configurations, we can evaluate the effectiveness of continuous repairing from a previous patch. Our findings indicate that holding restarting repair times constant while increasing continuous repair attempts leads to enhanced outcomes. As is shown in Table 8, when setting m to 40 and n to 1, the tool is able to generate 39 plausible fixes and 26 correct fixes. However, when m is kept constant and n is increased to 2, the tool's performance improves significantly, with plausible fixes increasing by 38.46% and correct fixes increasing by 26.92%. Furthermore, if increasing m from 2 to 3, *ContrastRepair* can perform better by achieving 11.11% more plausible fix and 12.12% more correct fix. Such a substantial advancement illustrates that the strategy of "Continuing from Previous Patch" is highly beneficial for enhancing the performance. Additionally, we also observe that the approach of slightly increasing depth (40×3) demonstrates superior performance compared to that of increasing breadth (40×3 (with the same prompt)). Therefore, it is crucial to find an optimal balance between m and n if the total number of queries is fixed. To explore this issue, we considered three distinct configurations: 12×10 , 40×3 , and 120×1 . Each configuration adheres to the same overall budget limit, totaling 120 tries. However, they differ in their approach: the 12×10 setup emphasizes a deeper search, the 120×1 aims for a broader search, while 40×3 strikes a balance between the two.

An analysis of the results of *ContrastRepair* in Table 9 reveals that the 40×3 configuration excels in terms of the number of correct/plausible fixes (37/60) and fewer queries (63.09). Conversely, 12×10 exhibits the least effectiveness (33/52). This disparity suggests that the parameter n is pivotal in influencing *ContrastRepair*'s performance. A larger n seems detrimental: if an initial repair attempt deviates from the optimal solution, realigning to a correct path becomes challenging as iterations increase. This can hinder the generation of accurate patches. However, setting n too low

Table 9. Evaluation on Restarting/Continuous Repair. Bold values indicate the best performance among the three LLM-based methods with corresponding settings.

Settings		BaseChatGPT			ChatRepair			ContrastRepair		
Iteration (m)	Round (n)	#Correct	#Plausible	#Query	#Correct	#Plausible	#Query	#Correct	#Plausible	#Query
40	3	31	48	76.16	33	54	69.09	37	60	63.09
12	10	25	39	86.33	31	50	73.23	33	52	70.6
120	1	30	50	73.69	33	55	68.22	35	58	65.08

can be counterproductive. If the initial direction of repair is on track but achieving the desired patch demands successive iterations, a moderately large n becomes beneficial. Continuous feedback in such cases can aid in formulating the correct fixes. Meanwhile, the results also indicate that under the other two $m \times n$ settings, *ContrastRepair* still outperforms the two baselines, it can outperform CHATREPAIR by fixing 6.45% and 6.06% more bugs on 12×10 and 120×1 .

Answer to RQ3: Utilizing test pairs provides more comprehensive information to LLMs for effective and efficient bug repair compared to solely relying on failed tests. The balance between restarting and continuous repairs is crucial to ensure the repair performance.

4.5 RQ4: Ablation Study on *ContrastRepair*

In this section, we delve into an ablation study to evaluate the usefulness of two critical components in *ContrastRepair*: pair selection and dependent functions. Specifically, we configure three variants of our tool: *ContrastRepair* has neither passing nor failing test cases (referred to as *w/o Test*), the tool that replaces the similarity-based test pair selection with random selection (referred to as *w/o Similarity*), the tool removes the process of type-aware mutation (referred to as *w/o Mutation*), and *ContrastRepair* without dependent functions (referred to as *w/o Context*) where the context refers to the functions with dependencies, such as caller or callee, with the buggy function in the project. The dependency relations can be extracted from the error trace information.

To evaluate the evaluation of *w/o Test* and *w/o Similarity*, we employ the same dataset used in RQ3, consisting of 106 bugs, each with multiple test pairs. For *w/o Context*, we chose all the 103 bugs from Defects4J where each sample has multiple dependent functions. The reason for this selection is that not each sample has the function invocation or test pair. For *w/o Mutation*, we select all the 45 bugs in which new test case pairs could be constructed via mutation. Similar to RQ3, we repair the bugs in a practical setting, i.e., single-function fix. Moreover, to mitigate the effect of randomness, we repeated the bug rectification process three times for each bug.

As indicated in Table 10, the results underscore the importance of test pair, dependent functions and the similarity-based selection used in *ContrastRepair*. For the 106 bugs, *ContrastRepair* generates 60.33 plausible fixes and 37.00 correct fixes on average. *ContrastRepair w/o Test* only has 53.33 plausible fixes and 32.33 correct fixes on average. However, when we replace the similarity selection with the random selection that may not select the pair with high similarity, the results are 56.33 plausible fixes and 33.33 correct fixes on average, which is higher than the setting without pairs but lower than *ContrastRepair*. This highlights an enhancement achieved by including the similarity-based pair selection. For the 103 bugs where *ContrastRepair* leverages the contextual information regarding dependent functions, it produces 64.67 plausible fixes and 36.00 correct fixes on average. Conversely, *w/o Context*, without the contextual information, generates 61.67 plausible fixes and 32.67 correct fixes. We conducted experiments to evaluate the effectiveness of mutation operations, focusing on 45 bugs that could be mutated to generate contrast test cases. The experimental results are presented in the last row of Table 10. For these bugs, *ContrastRepair*, when utilizing newly

Table 10. Effectiveness of Ablation on Different Components. Bold values indicate the best performance under different ablation settings.

Tools	#Correct	#Plausible	#Query
w/o Test	32.33	53.33	73.36
w/o Similarity	33.33	56.33	72.62
<i>ContrastRepair-106</i>	37.00	60.33	63.09
w/o Context	32.67	61.67	62.17
<i>ContrastRepair-103</i>	36.00	64.67	58.28
w/o Mutation	16.67	26.00	49.93
<i>ContrastRepair-45</i>	19.00	29.33	43.65

Table 11. Pass@m for Ablation on Similarity and Context. Bold values indicate the best performance under different ablation settings.

Tools	Pass@40		Pass@30		Pass@20		Pass@10		Pass@5	
	#C	#P	#C	#P	#C	#P	#C	#P	#C	#P
w/o Test	32.33	53.33	30.67	48.67	29.00	45.67	25.00	38.33	23.33	36.67
w/o Sim.	33.33	56.33	32.00	51.33	27.67	43.67	23.33	37.67	22.33	32.00
<i>ContrastRepair (106)</i>	37.00	60.33	36.00	59.00	33.67	55.33	31.67	50.00	28.67	42.33
w/o Con.	32.67	61.67	32.00	59.67	29.00	54.33	24.67	46.00	20.00	31.33
<i>ContrastRepair (103)</i>	36.00	64.67	34.67	61.33	33.00	58.67	30.33	52.00	25.33	41.33

constructed test case pairs derived from mutation, averages 29.33 plausible fixes and 19.00 correct fixes. In contrast, *ContrastRepair* without mutation, which handles these bugs without contrast test cases, averages only 26.00 plausible fixes and 16.67 correct fixes. Additionally, we observe that average query times have increased by 14.39% compared to *ContrastRepair* without mutation. These findings suggest that our mutations can generate informative pairs, leading to improved repair performance with fewer query attempts.

To demonstrate the value of each component, we adopted an additional metric, Pass@m, with m indicating the maximum number of repair attempts allowed (refer to Algorithm 1). We set varying values for m , as detailed in Table 11. In this table, *ContrastRepair* (106) and *ContrastRepair* (103) denote the results achieved by *ContrastRepair* on various datasets, which contained 106 and 103 bugs, respectively. Overall, the results continue to affirm the usefulness of test pairs, similarity-based test pair selection, and the context involving dependent functions. Notably, with a smaller m (i.e., a tighter budget), the impact of these components on generating correct (#C) and plausible (#P) patches becomes more pronounced, as seen in Pass@20, Pass@10, and Pass@5. Furthermore, the incorporation of similarity-based pair selection and contextual information contributes to a reduction in the number of queries made to ChatGPT API. For instance, on the three selected datasets, the average number of queries per bug for *ContrastRepair* is 63.09, 58.28 and 43.65, respectively. In contrast, *w/o Test*, *w/o Similarity*, *w/o Context*, and *w/o Mutation* have averages of 73.36, 72.62, 62.17, and 49.93, representing increases of 16.28%, 15.11%, 6.67%, and 14.39%, respectively.

Answer to RQ4: Incorporating similarity-based pair selection and dependent functions in prompts helps enhance the capabilities of *ContrastRepair* for bug repair. These components not only significantly improve the effectiveness of LLMs in generating more correct fixes but also reduce the number of API calls.

Table 12. Pass@m to Investigate the Effectiveness of Different Similarity Strategies to Select Cases. Bold values indicate the best performance among all the tools.

Tools	Pass@40		Pass@30		Pass@20		Pass@10		Pass@5	
	#C	#P	#C	#P	#C	#P	#C	#P	#C	#P
BM25	35.67	59.00	33.00	55.33	33.33	53.33	27.33	45.67	24.67	38.00
CodeBERT	33.00	57.33	32.67	53.00	29.00	51.00	27.67	45.00	26.33	38.00
UniXcoder	32.67	54.00	31.33	51.33	28.00	45.33	27.33	39.67	26.33	36.00
<i>ContrastRepair (106)</i>	37.00	60.33	36.00	59.00	33.67	55.33	31.67	50.00	28.67	42.33

Table 13. Evaluation on Similarity Score and Performance

Pair Selection	#Correct	#Plausible	#Query
Highest score (0.83)	27	44	41.88
Medium score (0.66)	25	41	45.1
Lowest score (0.56)	25	41	46.72

5 Discussion

5.1 Selection with Different Similarity Strategies

To investigate the effectiveness of our selection strategy, we use BM25 [57] (lexical-based) and CodeBERT [16], UniXcoder [19] (semantic-based) approaches for comparison. Compared with our used *Damerau-Levenshtein distance* for similarity measurement, BM25 utilized sparse vector representation for lexical matching. CodeBERT and UniXcoder are both learning-based approaches where CodeBERT learns code representations by the masked language modeling from a large code corpus, and UniXcoder further incorporates AST with different pre-training tasks for learning. Like the configuration in RQ4, we explore the effectiveness of the 106 bugs with multiple pairs. The experiments are repeated three times to avoid randomness. Furthermore, for CodeBERT and UniXcoder, we feed the test cases to the model and obtain the vector representation to calculate the similarity based on the cosine function.

The experimental results are presented in Table 12, and we can find that using *Damerau-Levenshtein distance* is superior to other approaches. As the objective of similarity measurement is finding the critical changes in the test cases, hence using the vector representation for retrieval may be indirect and inappropriate for this task rather than string-based similarity retrieval. The results in Table 12 have demonstrated the effectiveness of our similarity measurement approach.

5.2 Relation between Similarity Score and Performance

To explore the relationship between similarity scores and repair performance, we selected a subset of 60 bugs from the original 106, each containing at least three test case pairs, for our experiments. The settings were consistent with those used in *ContrastRepair*. The experimental results are shown in Table 13. Highest score, medium score, and lowest score refer to the three test pairs with the highest, medium, and lowest similarity scores for each buggy function. Their average scores are 0.83, 0.66, and 0.56, respectively. We observed that the repair performance is optimal when selecting highest score, with the number of correct and plausible fixes increasing by 8% and 7.32%, respectively. However, there is no significant difference in repair performance between medium score and lowest score as both their scores are not high enough. These results suggest that selecting pairs with higher similarity scores enhances repair performance.

Table 14. Contribution of Different Components. Bold values indicate the best performance among the three LLM-based methods on different subset.

Dataset	#Bugs	BaseChatGPT		ChatRepair		ContrastRepair	
		#Correct	#Plausible	#Correct	#Plausible	#Correct	#Plausible
Defects4J (pair)	106	36	50	45	63	51	71
Defects4J (context)	65	21	31	22	35	30	45
Defects4J (single)	164	39	62	57	80	62	85
Defects4J	335	96	143	124	178	143	201
Bears (pair)	50	7	16	9	21	13	25
Bears (context)	5	1	1	1	2	2	4
Bears (single)	8	1	4	1	4	1	4
Bears	63	9	21	11	27	16	33

5.3 Contribution of Different Components

Defects4J comprises 335 bugs, of which 106 bugs have associated test case pairs and 103 bugs possess contextual information usable in prompts, and there are 164 remaining bugs that lack both paired test cases and contextual information. To deal with these bugs, we replace test pairs by providing only one single failing test case. Additionally, we have incorporated another benchmark, Bears [45], to provide a more comprehensive analysis of the repair performance across various subsets. Specifically, Bears contains 63 bugs, with 50 having pairs, 5 with context, and 8 consisting of single test cases. The results are detailed in Table 14.

In Defects4J, *ContrastRepair* successfully repairs 19 more bugs than CHATREPAIR. Of these 19 bugs, six are from the Pair subset, and eight are from the Context subset. This indicates that the integration of pair and context information in *ContrastRepair* significantly enhances the LLM's repair capabilities. Together, these two factors contribute to 14 additional bug fixes, accounting for 73.68% (14/19) of the total improvement. We also observe that providing only a single failing test case can assist ChatGPT generate five more correct fix. This is attributable to the fact that CHATREPAIR only provides the run-time error message associated with the failing test without offering the complete details of the test case. By supplementing this information, our approach could allow ChatGPT to more effectively comprehend the underlying cause of the bug. Similarly, *ContrastRepair* also achieves more accurate fixes in the pair and context subsets in Bears, demonstrating the general applicability of our approach across different benchmarks.

5.4 Performance without Perfect Fault Localization

One limitation of our approach is that we primarily follow common settings in APR (e.g., CHATREPAIR [70], Coconut [42], and Cure [25]) and consider three repair scenarios: single-line, single-hunk, and single-function. Among these scenarios, both single-line and single-hunk are provided with perfect fault localization, leading to a higher correct rate than that on single-function. However, in some cases, the test pairs can implicitly aid fault localization by encouraging the LLM to understand the root cause of the bug (see Figure 1). In the future, we plan to evaluate the effectiveness of *ContrastRepair* by either removing the perfect fault localization (i.e., allowing LLMs to implicitly localize the fault) or by incorporating existing imperfect fault localization techniques, such as the fault localization tool Gzoltar, as used in ITER [77]).

5.5 Effectiveness in Multi-Location Repair Scenario

Multi-location bug repair remains a challenging task, the bugs typically involve multiple buggy code snippets dispersed across various functions and files. In fact, our method could also be used in

multi-location bugs. Specifically, we need to include the following components in the prompt: (1) all the buggy functions and their corresponding fault localization information; (2) dependent functions of these buggy functions; (3) contrastive test pairs derived from different buggy functions. However, relying solely on this information may not yield good repair outcomes due to the complexity of this task. To enhance the repair performance, we could improve *ContrastRepair* from the following aspects: (1) providing more detailed contextual information, such as dependent files which the buggy files depend on; (2) optimizing our search algorithm involves establishing more precise feedback to determine when to employ depth search versus breadth search; (3) providing information about the execution paths of test cases during runtime to help the LLM better understand the root cause triggering the bug more accurately. We plan to adapt and enhance our method for multi-location repair in our future work.

5.6 Threats to Validity

The first potential threat to the validity of our results is the selection of the datasets used for evaluation. The improvement obtained by *ContrastRepair* may not generalize to other repair datasets. To mitigate this threat, we have selected diverse datasets for evaluation, including Defects4J 1.2, Defects4J 2.0, and QuixBugs. The second threat is data leakage which arises when the ground truth patches are part of the original training data of ChatGPT. To address this, we selected an additional dataset, HumanEval-Java, which was published after the training of the ChatGPT models.

Manual verification of the plausible patches is a threat. A careful examination is needed to determine whether they are semantically equivalent. To mitigate the threat, we have invested significant labor costs to ensure the accuracy and impartiality of manual verification by inviting three researchers in SE field to check, respectively, each dedicating over 15 hours to manually validate the patches. They then discuss the patches where validation answers were inconsistent, ultimately reaching a consensus. The criterion for a correct patch is that it must either be identical to the ground truth or semantically equivalent. This approach to manual verification is consistent with methods used in several previous studies [67, 70, 75]. Additionally, we have made our patches open-source for public evaluation. The assessment of efficiency in our experiments could potentially present challenges. Notably, we did not measure the runtime of different methods due to two reasons. The main one is we have observed that LLM-based repair methods not only consume significantly less time but also perform much better compared to learning-based repair methods during the experimental process in RQ1. Even if *ContrastRepair* consumes about twice the total time (consists of query and mutation phases) as much as ChatRepair does (only contains query), it still remains entirely within an acceptable range. At this juncture, we think prioritizing repair effectiveness more than efficiency could be more reasonable when considering a tradeoff. Another one is the presence of numerous uncontrollable factors when invoking ChatGPT API calls, such as network delays, the configuration of ChatGPT models, and their parallelism and scheduling strategies. To mitigate this, we metered the number of API calls (i.e., *#Query*) to gauge the informativeness of prompts, which helps measure whether ChatGPT better understands the root cause of the bugs. Our observations suggest that the performance of conversation-based repair typically improves with an increased number of API calls. By comparing the query counts, we can assess the effectiveness of different feedback in guiding the repair process.

Moreover, the uncertainty inherent in the experimental setup poses a potential threat to the reproducibility of our results. Key factors of this uncertainty include the choice of random seeds and temperature settings, the selection of LLMs, and the non-deterministic nature of LLM inference. For instance, minor precision errors in floating-point operations can accumulate over time, potentially leading to significant variations in outcomes.

Another potential threat to the validity of our results is that some of the results from RewardRepair and TBar are collected from existing paper [70]. However, we believe that these comparisons do

not affect our conclusions, as there is a significant performance gap [70] between these methods and the LLM-based methods.

6 Related Work

6.1 APR

APR tools are employed to create patched code from the original code and the corresponding buggy location. Every patch generated by the APR tool undergoes validation against the test suite. Plausible patches refer to those that successfully pass the entire suite, while correct patches are plausible patches that effectively rectify the underlying bug. Generally speaking, there are two kinds of methods in the APR field, traditional and learning-based methods.

Traditional tools can be broadly divided into three main categories: heuristic-based [31, 32, 64], constraint-based [30, 41, 48], and template-based [35, 36, 47]. However, these traditional methods have some limitations. Template-based tools have been regarded as state-of-the-art ones among traditional methods due to their best repair performance, but these APR tools depend much on manually designed templates or specific fix patterns to repair limited kinds of bugs.

With the rapid development of DL techniques, there has been a growing focus on learning-based approaches, such as CURE [25], RewardRepair [76], Recoder [84], CoCoNut [42], SelfAPR [75], and ITER [77], which convert APR to Neural Machine Translation problem and have shown remarkable potential for enhancing bug repair performance. Compared to traditional methods, these learning-based methods can achieve better performance. And ITER can even achieve a state-of-the-art performance in multi-location repair scenarios, which is a significantly harder task in APR. Unlike conventional methods, learning-based approaches can automatically capture semantic relations among parallel bug-fixing pairs. This capability enables the creation of patch solutions that are not only more effective but also contextually aware. Nevertheless, the quality and quantity of the training datasets largely determine the effectiveness of the model. The training corpus may include a lot of noise when scraping repositories that have irrelevant commits and changes from GitHub.

Recently, researchers have explored the feasibility of employing potent LLMs for APR. LLMs demonstrate the ability to produce accurate patches directly from the contextual information without the necessity for fine-tuning. AlphaRepair [68] is the first cloze-style APR approach to directly leveraging large pre-trained code models for APR without any fine-tuning/retraining on historical bug fixes.

Despite LLM-based methods have yielded remarkable results [15, 24, 56, 60, 67, 68], they mainly focus on the buggy code and treat bug repair as a one-step process, overlooking the interactive and collaborative aspects inherent in bug resolution. Furthermore, by mining and analyzing historical data from interaction records with LLMs, and understanding the fundamental reasons for LLM failure in bug fixing, researchers can better guide LLMs to perform program repair in the correct direction in subsequent interactions.

Previous research [18, 71] has also demonstrated the effectiveness of applying test cases to code repair tasks, aiding APR tools in generating patches of high quality. Based on these intuitions, CHATREPAIR [70], the first fully automated conversation-driven APR approach that interleaves patch generation with instant feedback from test suites to perform APR in a conversational style, is proposed. Compared to traditional and learning-based methods, CHATREPAIR can achieve promising results through the use of LLMs by providing error feedback information for the conversations. Regrettably, this feedback may not consistently provide the precise and informative prompts required for efficient repairs. This article introduces a method, *ContrastRepair*, which aims to craft more specific and directive prompts for enhancing the capabilities of LLMs in accurately comprehending buggy-related semantics and generating high-quality patches.

6.2 LLMs

Recent advancements in generative AI have resulted in a significant increase in performance and the widespread embrace of LLMs [9, 83]. In the **Natural Language Processing (NLP)** domain, LLMs have achieved impressive performance in many tasks such as machine translation [61], text summarization [40], and classification [74]. Initially excelling in NLP tasks [50], such as document classification [22], text summarization [73], and machine translation [81]. Tremendous progress has been made by treating programming language as another sort of natural language and training LLMs on corpora of both program code and natural language text, and LLMs are now widely applied in various software engineering tasks including code generation [34, 80], code summarization [1, 38], code review [20], code completion [21], program specification generation [44], and commit message generation [39]. Since LLMs are designed to be general and capable of acquiring knowledge across various domains, researchers can then make use of LLMs for related downstream tasks by offering customized prompts or, if desired, a few task-solving demonstrations as input [37].

LLMs are advanced language models with massive parameter sizes and exceptional learning capabilities. The core module behind many LLMs such as GPT-3 [17], InstructGPT [54], and GPT-4 [53] is the self-attention module in Transformer [62] that serves as the fundamental building block for language modeling tasks. An important characteristic of LLMs is their ability to perform in-context learning [5], wherein the model is trained to produce text in accordance with a provided context or prompt. This capability allows LLMs to produce responses that are more coherent and contextually appropriate, rendering them well-suited for interactive and conversational purposes.

Reinforcement Learning from Human Feedback [12, 85] represents another pivotal element of LLMs. This approach entails fine-tuning the model by utilizing human-generated responses as rewards, enabling the model to learn from mistakes and enhance its performance progressively. Significantly, ChatGPT [52], with its emphasis on conversations and its capability to remember and refer back to previous dialogues, has attained state-of-the-art performance in diverse SE tasks [14, 43, 44, 60]. In this article, our purpose is focusing on how to design more powerful prompts by harnessing and utilizing information from test cases to prompt and inspire ChatGPT to understand the semantic aspects of bugs, achieving more effective bug repair, during the process of interacting and collaborating.

6.3 Automated Test Generation

Automated test generation is a widely used technique for detecting software defects by generating tests automatically. Techniques like fuzz testing [23, 49, 72] in black-box test generation involve supplying the **System Under Test (SUT)** with random test inputs (e.g., random bytes) without examining its internal code structure. Traditional black-box techniques can mainly be categorized into generation-based [23, 72] and mutation-based [8, 51, 66] ones. Generation-based techniques focus on creating test inputs from scratch, whereas mutation-based approaches introduce systematic alterations to existing seed inputs to craft a wider array of tests. A core drawback, often termed as “blindness,” of black-box strategies is their struggle to develop intricately designed test cases that thoroughly investigate complex code pathways. Generation-based techniques focus on directly generating test inputs from scratch, whereas mutation-based techniques apply systematic mutation changes on seed inputs to generate more diverse tests.

As a fundamental limitation of “blindness,” black-box methods may hardly produce well-designed test cases to exhaustively explore deep code paths. White-box methods yield higher-quality test cases by examining the source code of SUT. For example, symbolic execution [7, 27] overcomes coverage limitations by employing symbolic path constraint to generate tests that target deeper paths. However, due to the high cost of constraint solving, some solutions to enhance scalability have

been introduced, including algorithmic approaches like concolic testing [46, 58] and implementation strategies such as optimized compilation [55]. Serving as an intermediary between black and white-box testing, coverage-guided fuzzing [59, 78], also known as grey-box testing, leverages coverage information from SUT as feedback to adjust the process of generating and mutating inputs.

Due to our intention not to increase code coverage or the diversity of test cases, we continue to follow black-box test generation. Inspired by type-aware operator mutation [66], we implement mutations on parameters, which is referred to as type-aware parameter mutation. This strategy serves two purposes: first, it enables the rapid generation of a substantial number of test cases in a short period. Second, by employing fine-grained mutations that make minimal alterations to the original cases, we ensure that the newly generated cases are highly similar to their predecessors.

6.4 Test Case Similarity

To select similar pairs of test cases, we require certain metrics to measure the similarity of each pair. There are two widely used metrics: *lexical-based similarity* and *semantic-based similarity*. BM25 [57] is a lexical-based similarity which uses sparse vector representation for lexical matching. BM25 converts each code snippet as a bag-of-words representation and computes a lexical similarity between a pair $\langle F_i, P_j \rangle$. The computed similarity score is represented as $f_\phi(F_i, P_j) = \text{BM25}(F_i, P_j)$. As a sparse term-based metric, BM25 is sensitive to the choice of identifier naming in source code which does not impact the code semantics. **Dense Passage Retriever (DPR)** [26] belongs to semantic-based similarity. For training DPR to effectively learn code embeddings, both positive pairs and negative pairs are needed, and this is based on the assumption that the two similar code snippets often share similar semantics (e.g., identifiers and code structures). However, in this article, our test cases are not represented in the form of code snippets, they consist of specific values for the parameters within the buggy function instead. Consequently, the two aforementioned metrics used to assess code similarity are not applicable to our context. Our method involves first converting these values into strings and then calculating their similarity using the Damerau-Levenshtein distance [13, 33].

7 Conclusion

In this article, we introduced *ContrastRepair*, a novel conversation-based APR method that utilizes ChatGPT for repairing bugs. By generating contrastive test pairs, *ContrastRepair* provides informative feedback to ChatGPT, enabling better understanding of bug causes. We conducted extensive evaluations on diverse benchmark datasets, including Defects4J, QuixBugs, and HumanEval-Java, and compared *ContrastRepair* with state-of-the-art APR baselines. The results demonstrate that *ContrastRepair* achieves a new state-of-the-art in APR.

References

- [1] Toufique Ahmed and Premkumar Devanbu. 2022. Few-shot training LLMs for project-specific code-summarization. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 1–5.
- [2] Fatih Kadir Akin. 2023. The art of ChatGPT prompting. Retrieved from <https://fka.gumroad.com/l/art-of-chatgpt-prompting>
- [3] Tom Britton, Lisa Jeng, Graham Carver, and Paul Cheak. 2012. *Quantify the Time and Cost Saved Using Reversible Debuggers*. Technical Report. Cambridge Judge Business School.
- [4] Tom Britton, Lisa Jeng, Graham Carver, Paul Cheak, and Tomer Katzenellenbogen. 2013. *Reversible Debugging Software-Quantify the Time and Cost Saved Using Reversible Debuggers*. University of Cambridge.
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, Vol. 33, 1877–1901.
- [6] Volodymyr Buell. 2023. javaobj-py3. Retrieved from <https://pypi.org/project/javaobj-py3/>

- [7] Cristian Cadar, Daniel Dunbar, Dawson R. Engler, et al. 2008. Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, Vol. 8, 209–224.
- [8] Sang Kil Cha, Maverick Woo, and David Brumley. 2015. Program-adaptive mutational fuzzing. In *Proceedings of the 2015 IEEE Symposium on Security and Privacy*. IEEE, 725–741.
- [9] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Kaijie Zhu, Hao Chen, Linyi Yang, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2023. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology* 15, 3 (2024), 1–45.
- [10] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. arXiv:2107.03374. <https://arxiv.org/pdf/2107.03374>
- [11] Shigeru Chiba and Muga Nishizawa. 2023. Javassist. Retrieved from <https://www.javassist.org/>
- [12] Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep reinforcement learning from human preferences. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17)*. Curran Associates Inc., Red Hook, NY, 4302–4310.
- [13] Fred J. Damerau. 1964. A technique for computer detection and correction of spelling errors. *Communications of the ACM* 7, 3 (1964), 171–176.
- [14] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2024. Self-collaboration code generation via ChatGPT. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–38.
- [15] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated repair of programs from large language models. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1469–1481.
- [16] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. arXiv:2002.08155. Retrieved from <https://arxiv.org/pdf/2002.08155>
- [17] Luciano Floridi and Massimo Chiriatti. 2020. GPT-3: Its nature, scope, limits, and consequences. *Minds and Machines* 30 (2020), 681–694.
- [18] Xiang Gao, Sergey Mechtaev, and Abhik Roychoudhury. 2019. Crash-avoiding program repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 8–18.
- [19] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. Unixcoder: Unified cross-modal pre-training for code representation. arXiv:2203.03850. Retrieved from <https://arxiv.org/pdf/2203.03850>
- [20] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the potential of ChatGPT in automated code refinement: An empirical study. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 1–13.
- [21] Qi Guo, Xiaohong Li, Xiaofei Xie, Shangqing Liu, Ze Tang, Ruitao Feng, Junjie Wang, Jidong Ge, and Lei Bu. 2024. FT2Ra: A fine-tuning-inspired approach to retrieval-augmented code completion. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 313–324.
- [22] Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. 2023. Tabllm: Few-shot classification of tabular data with large language models. In *Proceedings of the International Conference on Artificial Intelligence and Statistics*. PMLR, 5549–5581.
- [23] Christian Holler, Kim Herzig, and Andreas Zeller. 2012. Fuzzing with code fragments. In *Proceedings of the 21st USENIX Security Symposium (USENIX Security'12)*, 445–458.
- [24] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of code language models on automated program repair. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1430–1442.
- [25] Nan Jiang, Thibaud Lutellier, and Lin Tan. 2021. Cure: Code-aware neural machine translation for automatic program repair. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1161–1173.
- [26] Vladimir Karpukhin, Barlas Öguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wenteau Yih. 2020. Dense passage retrieval for open-domain question answering. arXiv:2004.04906. Retrieved from <https://arxiv.org/pdf/2004.04906v2/1000>
- [27] James C. King. 1976. Symbolic execution and program testing. *Communications of the ACM* 19, 7 (1976), 385–394.
- [28] Jiaolong Kong. 2025. ContrastRepair. Retrieved from <https://github.com/kjl960913/ContrastRepair/tree/main>
- [29] Thomas D. LaToza, Gina Venolia, and Robert DeLine. 2006. Maintaining mental models: A study of developer work habits. In *Proceedings of the 28th international conference on Software engineering*, 492–501.
- [30] Xuan-Bach D Le, Duc-Hiep Chu, David Lo, Claire Le Goues, and Willem Visser. 2017. S3: Syntax-and semantic-guided repair synthesis via programming by examples. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 593–604.

- [31] Xuan Bach D. Le, David Lo, and Claire Le Goues. 2016. History driven program repair. In *Proceedings of the 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Vol. 1. IEEE, 213–224.
- [32] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2011. Genprog: A generic method for automatic software repair. *IEEE Transactions on Software Engineering* 38, 1 (2011), 54–72.
- [33] Vladimir I. Levenshtein et al. 1966. Binary codes capable of correcting deletions, insertions, and reversals. In *Soviet Physics Doklady*, Vol. 10, Soviet Union, 707–710.
- [34] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2023), 21558–21572.
- [35] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F. Bissyandé. 2019. Avatar: Fixing semantic bugs with fix patterns of static analysis violations. In *Proceedings of the 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 1–12.
- [36] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F. Bissyandé. 2019. TBar: Revisiting template-based automated program repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 31–42.
- [37] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys* 55, 9 (2023), 1–35.
- [38] Shangqing Liu, Yu Chen, Xiaofei Xie, Jingkai Siow, and Yang Liu. 2020. Retrieval-augmented generation for code summarization via hybrid GNN. arXiv:2006.05405. Retrieved from <https://arxiv.org/pdf/2006.05405>
- [39] Shangqing Liu, Yanzhou Li, Xiaofei Xie, Wei Ma, Guozhu Meng, and Yang Liu. 2024. Automated commit intelligence by pre-training. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–30.
- [40] Yang Liu. 2019. Fine-tune BERT for extractive summarization. arXiv:1903.10318. Retrieved from <https://arxiv.org/pdf/1903.10318>
- [41] Fan Long and Martin Rinard. 2015. Staged program repair with condition synthesis. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 166–178.
- [42] Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. 2020. Coconut: Combining context-aware neural translation models using ensemble for program repair. In *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*, 101–114.
- [43] Lezhi Ma, Shangqing Liu, Lei Bu, Shangru Li, Yida Wang, and Yang Liu. 2024. SpecEval: Evaluating code comprehension in large language models via program specifications. arXiv:2409.12866. Retrieved from <https://arxiv.org/pdf/2409.12866>
- [44] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. 2024. SpecGen: Automated generation of formal program specifications via large language models. arXiv:2401.08807. Retrieved from <https://arxiv.org/pdf/2401.08807>
- [45] Fernanda Madeiral, Simon Urli, Marcelo Maia, and Martin Monperrus. 2019. Bears: An extensible java bug benchmark for automatic program repair studies. In *Proceedings of the 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 468–478.
- [46] Rupak Majumdar and Koushik Sen. 2007. Hybrid concolic testing. In *Proceedings of the 29th International Conference on Software Engineering (ICSE'07)*. IEEE, 416–426.
- [47] Matias Martinez and Martin Monperrus. 2016. Astor: A program repair library for java. In *Proceedings of the 25th International Symposium on Software Testing and Analysis*, 441–444.
- [48] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. 2016. Angelix: Scalable multiline program patch synthesis via symbolic analysis. In *Proceedings of the 38th International Conference on Software Engineering*, 691–701.
- [49] Barton P. Miller, Lars Fredriksen, and Bryan So. 1990. An empirical study of the reliability of UNIX utilities. *Communications of the ACM* 33, 12 (1990), 32–44.
- [50] Bonan Min, Hayley Ross, Elior Sulem, Amir Pouran Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heintz, and Dan Roth. 2023. Recent advances in natural language processing via large pre-trained language models: A survey. *ACM Computing Surveys* 56, 2 (2023), 1–40.
- [51] Peter Oehlert. 2005. Violating assumptions with fuzzing. *IEEE Security & Privacy* 3, 2 (2005), 58–62.
- [52] OpenAI. 2023. ChatGPT. Retrieved from <https://chat.openai.com/>
- [53] OpenAI. 2023. GPT-4 Technical Report. Retrieved from <https://cdn.openai.com/papers/gpt-4.pdf>
- [54] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, Vol. 35, 27730–27744.
- [55] Sebastian Poeplau and Aurélien Francillon. 2020. Symbolic execution with {SymCC}: Don't interpret, compile!. In *Proceedings of the 29th USENIX Security Symposium (USENIX Security'20)*, 181–198.
- [56] Julian Aron Prenner, Hlib Babii, and Romain Robbes. 2022. Can OpenAI's codex fix bugs? An evaluation on QuixBugs. In *Proceedings of the 3rd International Workshop on Automated Program Repair*, 69–75.

- [57] Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.
- [58] Koushik Sen, Darko Marinov, and Gul Agha. 2005. CUTE: A concolic unit testing engine for C. *ACM SIGSOFT Software Engineering Notes* 30, 5 (2005), 263–272.
- [59] Kosta Serebryany. 2016. Continuous fuzzing with libfuzzer and addresssanitizer. In *Proceedings of the 2016 IEEE Cybersecurity Development (SecDev)*. IEEE, 157–157.
- [60] Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. 2023. An analysis of the automatic bug fixing performance of ChatGPT. arXiv:2301.08653. Retrieved from <https://arxiv.org/pdf/2301.08653>
- [61] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to sequence learning with neural networks. In *Proceedings of the 28th International Conference on Neural Information Processing Systems (NIPS'14)*. Vol. 2, MIT Press, Cambridge, MA, 3104–3112.
- [62] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17)*. Curran Associates Inc., Red Hook, NY, 6000–6010.
- [63] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 8696–8708.
- [64] Ming Wen, Junjie Chen, Rongxin Wu, Dan Hao, and Shing-Chi Cheung. 2018. Context-aware patch generation for better automated program repair. In *Proceedings of the 40th International Conference on Software Engineering*, 1–11.
- [65] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C Schmidt. 2023. A prompt pattern catalog to enhance prompt engineering with ChatGPT. arXiv:2302.11382. Retrieved from https://file.mixpaper.cn/paper_store/2023/681177f8-cd15-4e0f-a23b-997c6b9f9dd2.pdf
- [66] Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2020. On the unusual effectiveness of type-aware operator mutations for testing SMT solvers. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–25.
- [67] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *2023IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1482–1494.
- [68] Chunqiu Steven Xia and Lingming Zhang. 2022. Less training, more repairing please: Revisiting automated program repair via zero-shot learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. ACM, New York, NY, 959–971. DOI: <https://doi.org/10.1145/3540250.3549101>
- [69] Chunqiu Steven Xia and Lingming Zhang. 2023. Conversational automated program repair. arXiv:2301.13246. Retrieved from <https://arxiv.org/pdf/2301.13246>
- [70] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated program repair via conversation: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 819–831.
- [71] Jinqiu Yang, Alexey Zhikartsev, Yuefei Liu, and Lin Tan. 2017. Better test cases for better automated program repair. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 831–841.
- [72] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, 283–294.
- [73] Xianjun Yang, Yan Li, Xinlu Zhang, Haifeng Chen, and Wei Cheng. 2023. Exploring the limits of ChatGPT for query or aspect-based text summarization. arXiv:2302.08081. Retrieved from <https://arxiv.org/pdf/2302.08081>
- [74] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, RussR. Salakhutdinov, and Quoc V. Le. 2019. Xlnet: Generalized autoregressive pretraining for language understanding. In *Advances in Neural Information Processing Systems*, Vol. 32, 5753–5763.
- [75] He Ye, Matias Martinez, Xiapu Luo, Tao Zhang, and Martin Monperrus. 2022. Selfapr: Self-supervised program repair with test execution diagnostics. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 1–13.
- [76] He Ye, Matias Martinez, and Martin Monperrus. 2022. Neural program repair with execution-based backpropagation. In *Proceedings of the 44th International Conference on Software Engineering*, 1506–1518.
- [77] He Ye and Martin Monperrus. 2024. ITER: Iterative neural repair for multi-location patches. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 1–13.
- [78] M. Zalewski. 2018. American fuzzing lop (AFL). Retrieved from <https://lcamtuf.coredump.cx/afl/>
- [79] Andreas Zeller and Ralf Hildebrandt. 2002. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering* 28, 2 (2002), 183–200.
- [80] Zhengran Zeng, Hanzhuo Tan, Haotian Zhang, Jing Li, Yuqun Zhang, and Lingming Zhang. 2022. An extensive study on pre-trained models for program understanding and generation. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 39–51.

- [81] Biao Zhang, Barry Haddow, and Alexandra Birch. 2023. Prompting large language model for machine translation: A case study. In *International Conference on Machine Learning*. PMLR, 41092–41110.
- [82] Qunjun Zhang, Chunrong Fang, Yuxiang Ma, Weisong Sun, and Zhenyu Chen. 2023. A survey of learning-based automated program repair. arXiv:2301.03270. Retrieved from <https://arxiv.org/pdf/2301.03270>
- [83] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. arXiv:2303.18223. Retrieved from https://paper-notes.zhjwpu.com/assets/pdfs/llm_survey_2303.18223.pdf
- [84] Qihao Zhu, Zeyu Sun, Yuan-an Xiao, Wenjie Zhang, Kang Yuan, Yingfei Xiong, and Lu Zhang. 2021. A syntax-guided edit decoder for neural program repair. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 341–353.
- [85] Daniel M Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. 2019. Fine-tuning language models from human preferences. arXiv:1909.08593. Retrieved from <https://arxiv.org/pdf/1909.08593>

Received 3 March 2024; revised 26 December 2024; accepted 4 February 2025